Asean Geo AI Fusion (AGAIF) 2026

Bootcamp 1 - Certified Assessment 3 Report

Reference code : MY-564

Name : Wong Yan Wen

Table of contents

1.0 AI tools and prompts used

1.1 AI tools :

Gemini 3.1 Pro - used in code generation and troubleshooting

1.2 Prompts used:

1.2.1 Prompt 1

This is a snippet of my Asean.js : <omitted Asean,js file content>
This is a snippet of my Places.js : <omitted Place.js file content>
I want to colour the polygons and markers according to their country, how do i modify my current code to be more concise ?

```
L.geoJSON(aseanJs, {
        style : function (features){
            // case 1 - 10
            //colour polygons based on country (object id)
            switch (features.properties.OBJECTID){
                case 1 : return {color : "#9e0142"};
                case 2 : return {color : "#d53e4f"};
                case 3 : return {color : "#f46d43"};
                case 4 : return {color : "#fee08b" };
                case 5 : return {color : "#fdae61"};
                case 6 : return {color : "#5e4fa2"};
                case 7 : return {color : "#abdda4"};
                case 8 : return {color : "#66c2a5"};
                case 9 : return {color : "#3288bd"};
                case 10 : return {color : "#e6f598"};
            }
        }
    }).addTo(map);
    L.geoJSON(placeJs).addTo(map);
```

1.2.2 Prompt 2

(omitted base code)
Based on my current code, Asean.js and Places.js, create a leaflet application that has similar output to the following picture:

1.2.3 Prompt 3

Why is the popup not showing up?

```
function onEachFeature(feature, layer) {
        // does this feature have a property named "name"?

        if (feature.properties && feature.properties.name) {
            //layer.bindPopup(feature.properties.name);
            pointToLayer.bindPopup(feature.properties.name);
        }
    }

    L.geoJSON(aseanJs, {style : getCountryStyle}).addTo(map);

    L.geoJSON(placeJs, {
           pointToLayer: function (feature, latlng) {
            //L.circleMarker replaces the default image pin
           //set the radius and pass in shared style function
              let markerParams = getCountryStyle(feature);
              markerParams.radius = 6; // Set the size of the point
              return L.circleMarker(latlng, markerParams);
      }
    }, { onEachFeature : onEachFeature}).addTo(map);
```

2.0 Dataset

The datasets used for this interactive web mapping application is provided by the organizers :

- Asean.cpg
- Asean.dbf
- Asean.prj
- Asean.sbn
- Asean.sbx
- Asean.shp
- Asean.shx
- Place.cpg
- Place.dbf
- Place.prj
- Place.sbn
- Place.sbx
- Place.shp
- Place.shp
- Place.shx

To briefly explain the file format:
.shp (Shape Format): holds the actual geometry data—the exact coordinate lines that draw the borders of the ASEAN countries and the points for the places.

.dbf (Database Application): a database table that holds the "attributes" or information about those shapes (e.g., country names, IDs, or regional data).

.prj (Projection Format): specifies the coordinate system the map uses so it aligns perfectly with the real world.

.shx, .sbn, .sbx (Index files): index files that link the geometry (.shp) to the database table (.dbf) so everything loads quickly.

.cpg (Code Page): defines the character encoding (like UTF-8) so text and names display correctly.

All the files that were named “Asean” were converted into a single GeoJSON file “Asean.geojson”, while all the files that were named “Place” were converted into a single GeoJSON file “Place.geojson”.
“Asean.geojson” contains information and coordinates needed to construct the polygon that best fits the country's geographical shape. “Place.geojson” contains information on cities, population and country.

2.1 Snippet of Asean.geojson:


```
{
"type": "FeatureCollection",
"name": "Asean",
"features": [
{ "type": "Feature", "properties": { "OBJECTID": 1, "Country":
"Singapore", "Flag":
"https://moe-edugis.maps.arcgis.com/sharing/rest/content/items/34a782
45cd504389b4637181afaee1ac/data" }, "geometry": { "type": "Polygon",
"coordinates": [ [ [ 103.9905363, 1.3832911 ], [ 103.9677452,
1.387218 ], [ 103.8940092, 1.4075 ], [ 103.8794093, 1.417218 ], [
103.8483093, 1.4347181 ], [ 103.8294183, 1.4436091 ], [ 103.8172003,
1.4452821 ], [ 103.8041361, 1.445 ], [ 103.7135913, 1.429445 ], [
103.6991362, 1.4236091 ], [ 103.6844003, 1.4111091 ], [ 103.6777452,
1.403609 ], [ 103.6710823, 1.3931911 ], [ 103.6433092, 1.331391 ], [
103.6409452, 1.318336 ], [ 103.6469183, 1.3046539 ], [ 103.7983092,
1.259445 ], [ 103.8419182, 1.259027 ], [ 103.9599723, 1.320836 ], [
103.9849723, 1.3441642 ], [ 103.9941363, 1.3552821 ], [ 103.9979452,
1.3698641 ], [ 103.9905363, 1.3832911 ] ] ] } },

<omitted rest of file>
```


2.2 Snippet of Place.geojson:


```
{
"type": "FeatureCollection",
"name": "Place",

"features": [
{ "type": "Feature", "properties": { "FID_Place1": 3030, "osm_id":
262314145, "name": "Kupang", "type": "city", "population": 0,
"FID_Asean": 9, "OBJECTID": 10, "Country": "Indonesia", "Lat":
-10.1632209, "Long": 123.6017755 }, "geometry": { "type": "Point",
"coordinates": [ 123.6017755, -10.1632209 ] } },

{ "type": "Feature", "properties": { "FID_Place1": 3090, "osm_id":
264994734, "name": "Ende", "type": "city", "population": 60000,
"FID_Asean": 9, "OBJECTID": 10, "Country": "Indonesia", "Lat":
-8.8439147, "Long": 121.647964 }, "geometry": { "type": "Point",
"coordinates": [ 121.647964, -8.8439147 ] } },

<omitted rest of file>
```

3.0 Development process

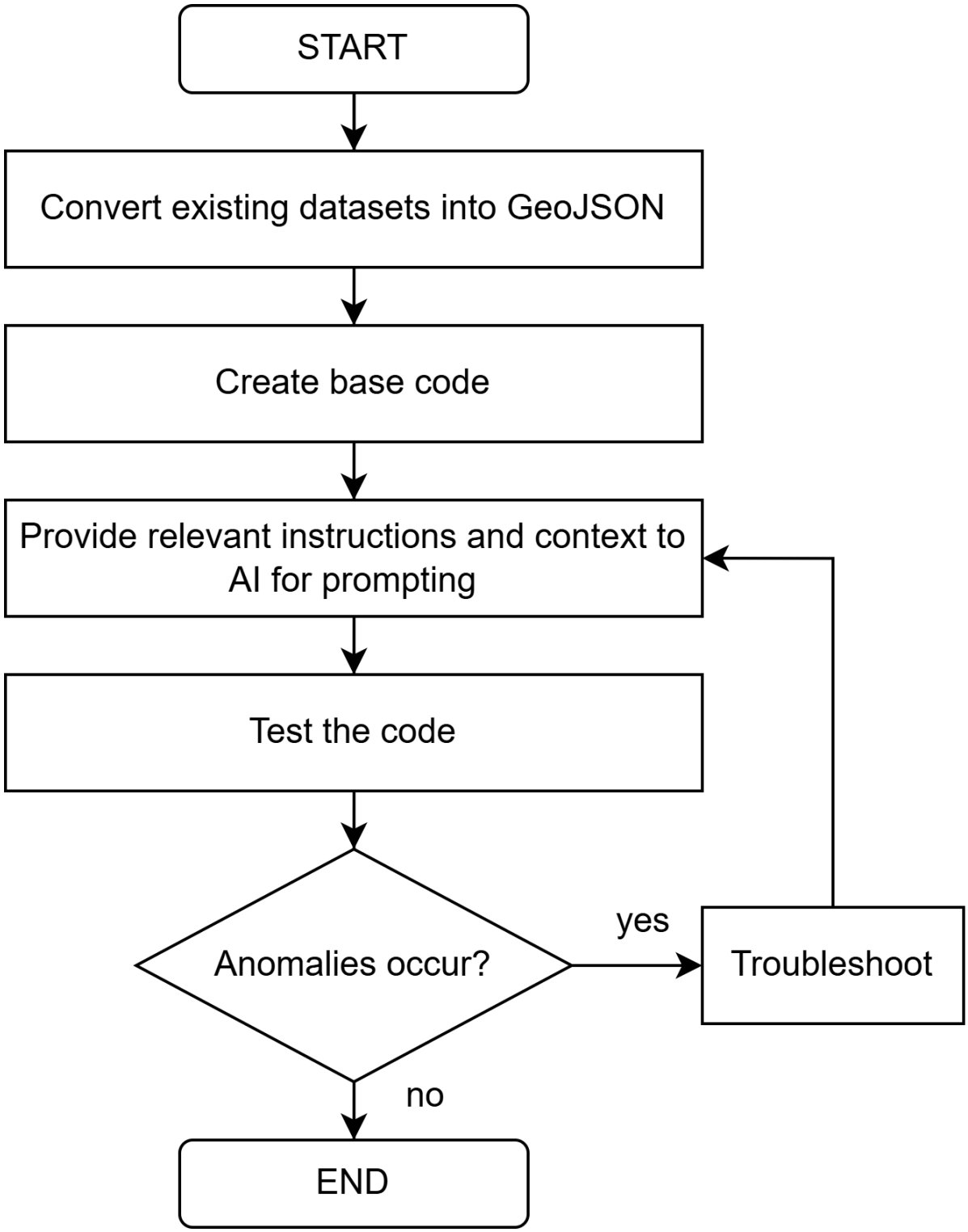

Image 3.0 developmental process of ai-assisted web mapping project

4.0 Improvement

4.1 Increased interactivity

Initially , the application will show all markers available in Place.js . When the markers are clicked, information of the marker will be displayed on screen as a popup.

4.1.1 Snippet of old code:

```
//function to get colours for each country
    function getCountryStyle(feature) {
        return {
            // Looks up the color based on OBJECTID. Falls back to
white if missing.
            fillColor: countryColors[feature.properties.Country] ||
"#ffffff",
            color: "#333333", // Dark gray outline for both polygons
and circles
            weight: 1,        // Outline thickness
            fillOpacity: 0.5 // Adjust transparency
        };
    }

    //draw country borders
    L.geoJSON(aseanJs, {style : getCountryStyle}).addTo(map);

    function onEachFeature(feature, layer) {
        // Look for 'name' (for places) OR 'Country' (for polygons)
        let popupContent = feature.properties.name ||
feature.properties.Country;

        if (popupContent) {
            layer.bindPopup(popupContent);
        }
    }

    // Render Asean Polygons
    L.geoJSON(aseanJs, {
        style : getCountryStyle,
        onEachFeature : onEachFeature
    }).addTo(map);
```

```
// Render Place Markers

 L.geoJSON(placeJs,{onEachFeature : onEachFeature }).addTo(map);
```

4.1.2 Sample output of old code:

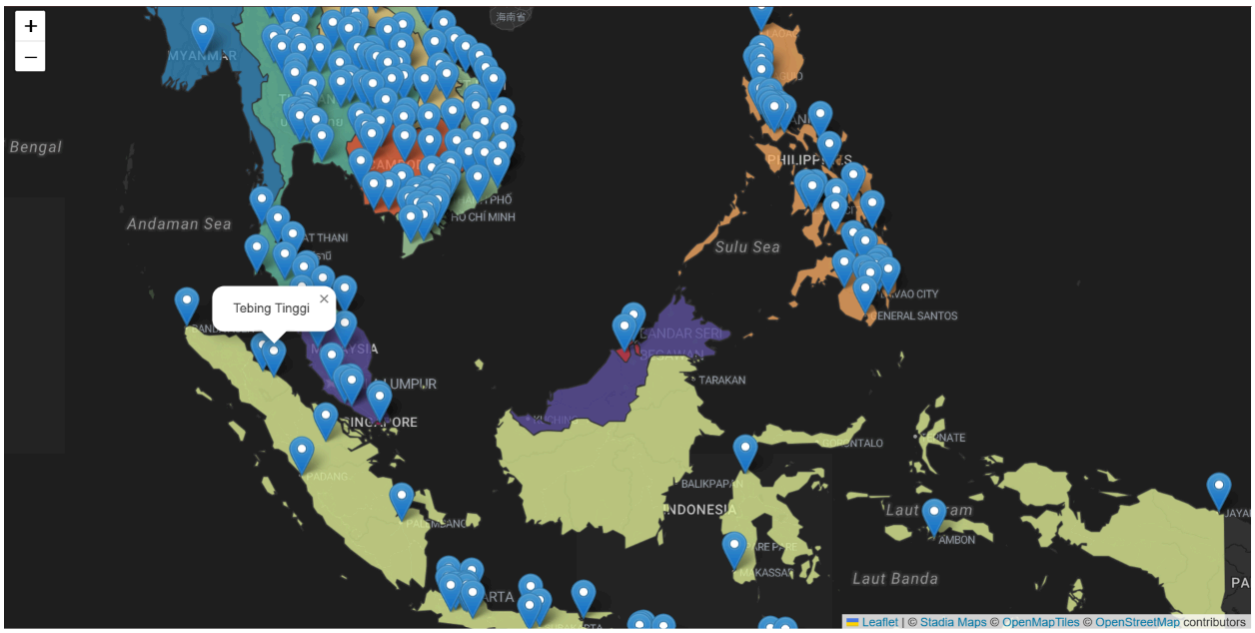

Image 4.1.2 shows sample output of old code

Using Gemini 3.1 Pro, interactivity of the web application has been increased. Instead of showing the name of the places when they are clicked, the information is displayed to the user right off the bat. Each country's markers have their own specified colours too. When the user clicks the markers, the coordinates and population of the place is shown.

The map defaults to showing Indonesia markers, however, users can select other countries to view places in each country. An “all countries” option is provided for the users to view all locations available. The number of locations is shown on the left-hand side of the screen.

In addition, users can search a place by name easily. The bottom right corner also includes a country legend to help users differentiate different countries by their colour. These changes greatly improves user experience and makes the map less cluttered.

4.1.3 Snippet of new code:

```
//draw country borders
    L.geoJSON(aseanJs, {style : getCountryStyle}).addTo(map);


    // 4. Variables to handle filtering logic
    let placesLayer;
    let currentFilter = "Indonesia"; // Defaults to Indonesia to
match the screenshot
    let searchQuery = "";
    let totalLocations = 0;

    // 5. Function to Render/Re-render Places based on filters
    function renderPlaces() {
        // Remove old layer(if exist) to make room for filtered one
        if (placesLayer) {
            map.removeLayer(placesLayer);
        }

        totalLocations = 0; // Reset count

        // 4. Add placesLayer
        placesLayer = L.geoJSON(placeJs, {
            //filter logic, check of it matches dropdown and
searchbox
            filter: function(feature) {
                        let matchesCountry = (currentFilter === "All"
|| feature.properties.Country === currentFilter);
                        //safe name check
                        let placeName = feature.properties.name ||
"";
                        let matchesSearch = (searchQuery === "" ||
placeName.toLowerCase().includes(searchQuery.toLowerCase()));
                        return matchesCountry && matchesSearch;
                    },

            //render colour of markers
            pointToLayer: function (feature, latlng) {
                //for each point in the country, increment
totalLocations
                totalLocations++;
                // L.circleMarker replaces the default image pin with
a colorable vector circle
```

```
                // We set the radius and pass in our shared style
function
                let markerParams = getCountryStyle(feature);
                markerParams.radius = 6; // Set the size of the point

                return L.circleMarker(latlng, markerParams);
            },
            //popup events
            onEachFeature : function (feature, layer) {
                let placeName = feature.properties.name || "Unknown
Location";

                // does this feature have a property named "name"?
                if (feature.properties.name) {
                    //use bindTooltip (permanent) instead of
bindPopup (temporary)
                    layer.bindTooltip(feature.properties.name, {
                        permanent: true,       // Always visible
                        direction: 'right',    // Show on the right
                        className: 'custom-label', // Uses custom CSS
                        offset: [5, 0]  // Shift slightly to the
right
                    });
                }

            // Popup(Shown when clicked, display coordinates + popul)
            let lat = feature.geometry.coordinates[1].toFixed(4);
            let lng = feature.geometry.coordinates[0].toFixed(4);
            let popul = feature.properties.population;
            layer.bindPopup(`<b>${placeName}</b>
                            <br>Coordinates: ${lat}, ${lng}
                            <br>Population: ${popul} `);

            }
        }).addTo(map);

        // Update the total count text in the UI panel
        let countDisplay = document.getElementById("location-count");
        if (countDisplay) countDisplay.innerHTML = `<b>Total
locations: ${totalLocations}</b>`;

    }

    // 6. Create the Top Left Locator Panel
    var locatorControl = L.control({position: 'topleft'});
```

```
    locatorControl.onAdd = function (map) {
        var div = L.DomUtil.create('div', 'custom-panel
locator-panel');

        // Build the HTML for the panel
        div.innerHTML = `
            <h4>ASEAN Places Locator</h4>
            <div id="location-count"><b>Total locations: 0</b></div>
            <div class="hint-text">
                Click a marker to view coordinates. Labels are shown
for each location.
            </div>
            <select id="country-filter">
                <option value="All">All Countries</option>
                <option value="Brunei Darussalam">Brunei
Darussalam</option>
                <option value="Cambodia">Cambodia</option>
                <option value="Indonesia" selected>Indonesia</option>
                <option value="Laos DR">Laos DR</option>
                <option value="Malaysia">Malaysia</option>
                <option value="Myanmar">Myanmar</option>
                <option value="Philippines">Philippines</option>
                <option value="Singapore">Singapore</option>
                <option value="Thailand">Thailand</option>
                <option value="Vietnam">Vietnam</option>
            </select>
                <input type="text" id="name-search"
placeholder="Search place name...">
                `;

            // Prevent map dragging when clicking inside the panel
            L.DomEvent.disableClickPropagation(div);
            return div;
        };

        locatorControl.addTo(map);

// 7. Create the Bottom Left Legend Panel
    var legendControl = L.control({position: 'bottomleft'});
    legendControl.onAdd = function (map) {
        var div = L.DomUtil.create('div', 'custom-panel');
        div.innerHTML = '<h4>Country Legend</h4>';

        // Loop process to generate HTML for each country
        for (let country in countryColors) {
```

```
            div.innerHTML += `
                <div class="legend-item">
                    <span class="legend-color"
style="background:${countryColors[country]}"></span>
                    <span>${country}</span>
                </div>
            `;
        }
            return div;
    };

    legendControl.addTo(map);

    // 8. Attach Event Listeners to our UI Panel Inputs
    // Dropdown listener

document.getElementById('country-filter').addEventListener('change',
function(e) {
            currentFilter = e.target.value;
            searchQuery = ""; //clear searchbox when changing country
            renderPlaces(); // Re-draw markers when changed

            // Optional: Auto zoom to fit the newly filtered dots
            if (totalLocations > 0) {
                map.fitBounds(placesLayer.getBounds(), { padding:
[50, 50] });
            }
        });

        // Text box search listener

document.getElementById('name-search').addEventListener('input',
function(e) {
            searchQuery = e.target.value;

            // If typing a search, set dropdown to "All Countries" so
it searches everywhere
            if (searchQuery !== "") {
                currentFilter = "All";
                document.getElementById('country-filter').value =
"All";
            }
            renderPlaces(); // Re-draw markers
        });
```

```
// 9. Final Step: Run the render function once when the page loads
        renderPlaces();

        // Auto-fit the map to show Indonesia based on the initial
load
        map.fitBounds(placesLayer.getBounds(), { padding: [50, 50]
});
```

4.1.4 Screenshot of working application:

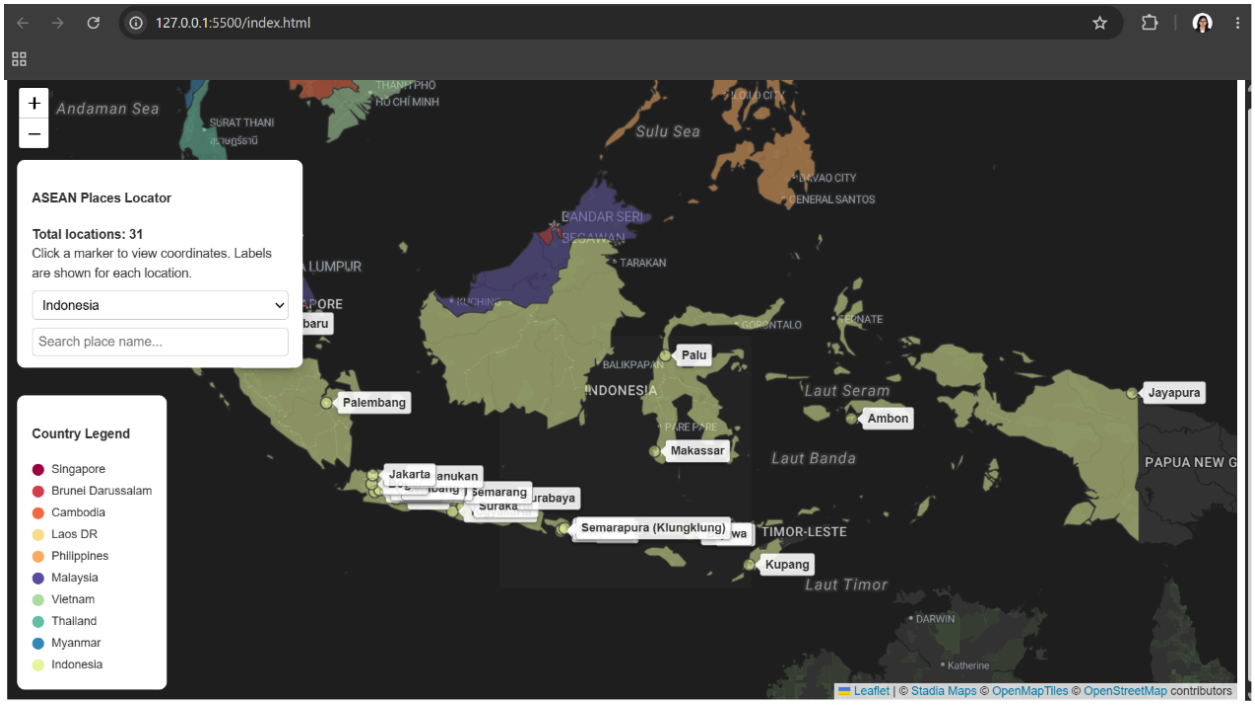

Image 4.1.4.1 shows the default view when user enters the application.

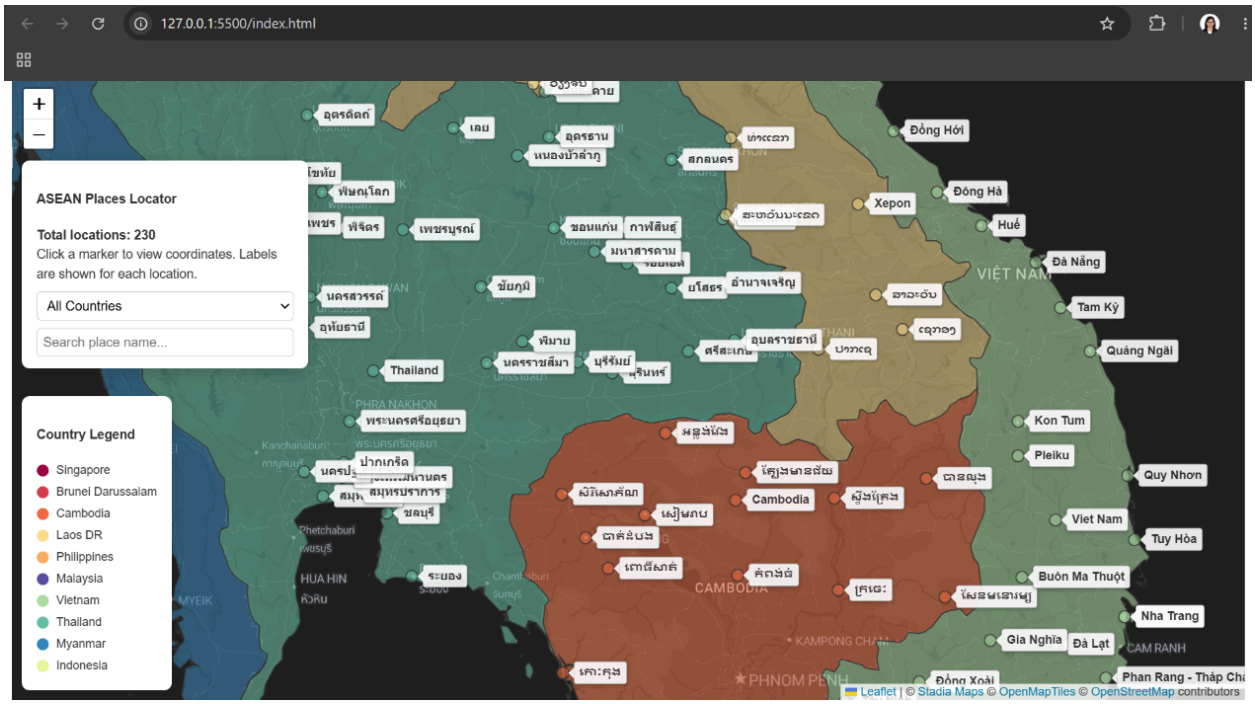

Image 4.1.4.2 shows close-up of markers on Thailand, Laos DR, Cambodia and Vietnam

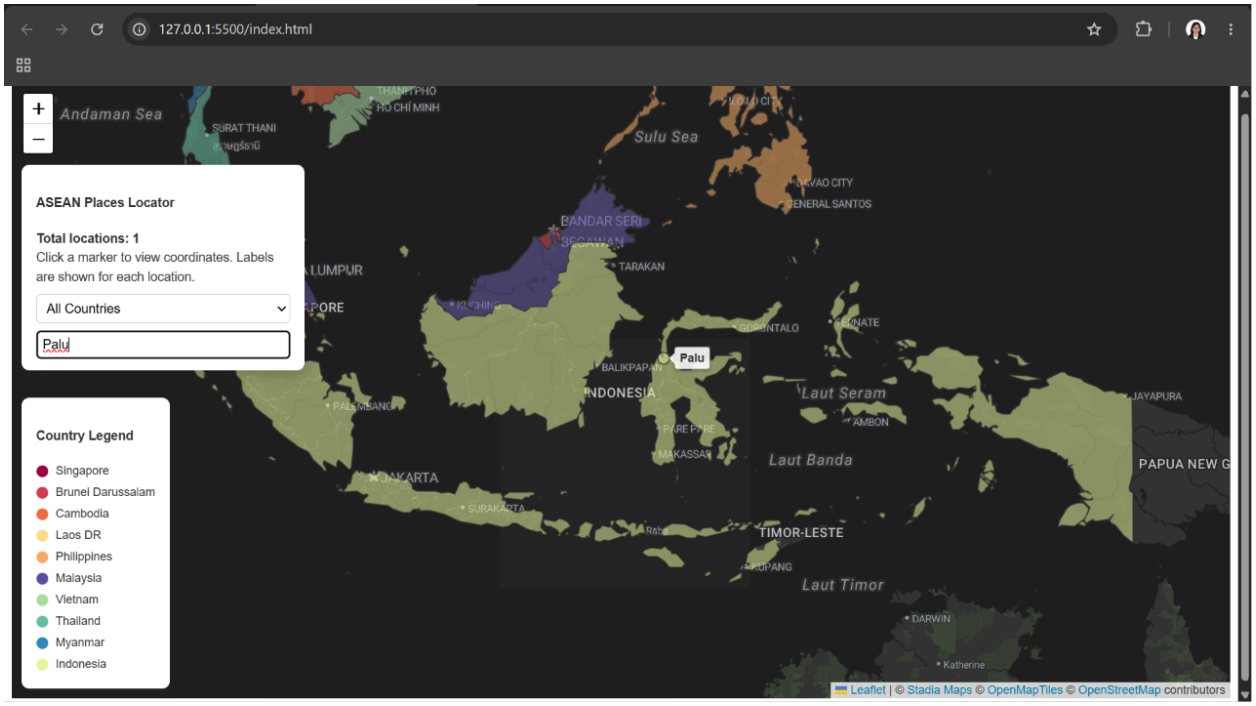

Image 4.1.4.3 shows the functionality of the search box

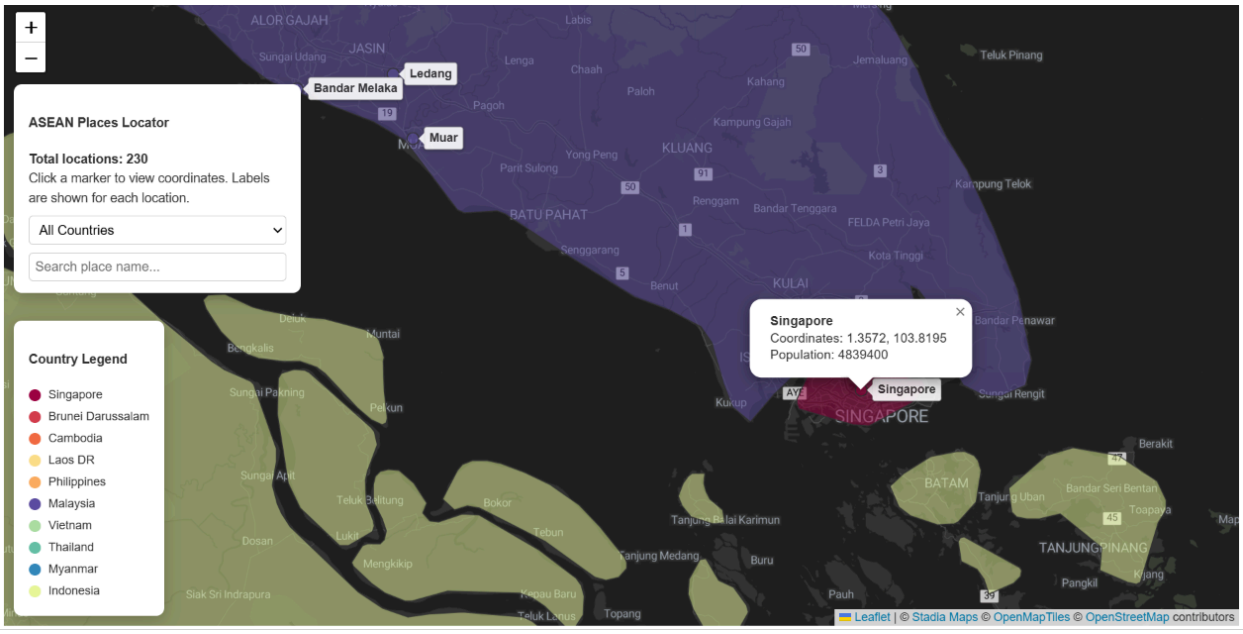

Image 4.1.4.4 shows popup containing coordinates and population of Singapore

5.0 Reflection

In conclusion, I have learnt a great deal about Leaflet.js. It was incredibly rewarding as this is a topic outside of my usual university syllabus and I get to explore this rich dataset. I have also learnt that UI design is a lot more complicated than I thought as I need to sync up user input, what's on display, with the dataset itself. All in all, I am very glad that I have the chance to make this web application come to life as it teaches me the basics of how geographical information is implemented in real world applications.

6.0 Appendix

Link to github repo:
https://github.com/wong-yanwen/AGAIF2026_BC1_CA3_MY-564_WONG_YAN_WEN_REPO.git

Link to web application:
https://wong-yanwen.github.io/AGAIF2026_BC1_CA3_MY-564_WONG_YAN_WEN_REPO/